🤖 AI System Design — AI-Based Lifestyle Intelligence Platform

Owner: Shaurya (Design) + Jagdish (Integration)

Approach: Hybrid — Rule-Based (Phase 1) + LLM-Powered (Phase 2) + RAG Chatbot (Phase 3)

---

1. Overview

The AI module enhances the platform by providing **data-driven insights and conversational assistance** based on user lifestyle and financial data.

The system follows a **hybrid approach**:

Phase	Type	Dependency	Availability
Phase 1	Rule-based analytics	None (runs in Spring Boot)	Always available
Phase 2	LLM-based insights	External AI service	Depends on AI service
Phase 3	RAG chatbot	AI service + Vector DB	Depends on AI service

Critical design rule: Phases 2 and 3 are **additive**. If the AI service is down, Phase 1 rule-based insights still work. The system never breaks.

---

2. Architecture

```
┌──────────────────────────────────────────────────────────┐
│                   Spring Boot Backend                    │
│                                                          │
│   User Request → Data Aggregation → AI Integration       │
│                                                          │
│    ┌──────────────────────────────────────────┐          │
│    │          AI Integration Module           │          │
│    │                                          │          │
│    │    ┌──────────────────────────────┐      │          │
│    │    │  Rule-Based Insight Engine   │      │          │
│    │    │  (Always available, Phase 1) │      │          │
│    │    └──────────────────────────────┘      │          │
│    │                                          │          │
│    │    ┌──────────────────────────────┐      │          │
│    │    │  AI Service Client           │──────┼──┐       │
│    │    │  (Calls external FastAPI)    │      │  │       │
│    │    └──────────────────────────────┘      │  │       │
│    └──────────────────────────────────────────┘  │       │
```

```
└──────────────────────────────────────────────┘   │
                                                   │
                                                   │
                                     REST API      │
                                                   ▼
                                  ┌──────────────────────┐
                                  │   AI Service         │
                                  │   (Python FastAPI)   │
                                  │                      │
                                  │   ┌──────────────┐   │
                                  │   │  LLM Engine  │   │
                                  │   └──────────────┘   │
                                  │                      │
                                  │   ┌──────────────┐   │
                                  │   │  RAG Pipeline│   │
                                  │   └──────────────┘   │
                                  │                      │
                                  │   ┌──────────────┐   │
                                  │   │  Vector DB   │   │
                                  │   └──────────────┘   │
                                  └──────────────────────┘
```

---

3. Data Flow

```
1. User requests insight or asks a question
        ↓
2. Backend fetches user data:
   • MySQL → user profile, expenses
   • MongoDB → daily logs, journals, habits
        ↓
3. Backend aggregates into summary format
   (AI never sees raw database records)
        ↓
4. Phase 1: Rule engine checks thresholds → generates deterministic insights
   Phase 2: Backend sends summary to AI service → LLM generates insights
   Phase 3: Backend sends summary + question → RAG pipeline responds
        ↓
5. Response returned to frontend
```

Key principle: The AI service **never accesses databases directly**. The backend is the gatekeeper.

---

4. Data Preparation Strategy

The backend aggregates raw data into structured summaries before any AI processing.

Example Aggregated Input:

```
{
  "userId": "101",
```

```
  "period": "weekly",
  "avgSleep": 5.5,
  "avgSteps": 3000,
  "avgWaterIntake": 2.1,
  "weeklySpending": 3200,
  "topExpenseCategory": "Food",
  "habitCompletionRate": 60,
  "moodDistribution": {
    "Happy": 3,
    "Tired": 2,
    "Neutral": 1,
    "Stressed": 1
  },
  "sleepTrend": "declining",
  "stepsTrend": "stable"
}
```

Why Aggregated Summaries?

Raw Data Problem	Aggregated Summary Solution
Too much noise for LLM	Clean, structured input
Privacy risk (raw journals)	Only statistical summaries
Token cost (large context)	Compact payload
Inconsistent structure	Standardized JSON format

---

5. Phase 1 — Rule-Based Insight Engine

Description

Simple threshold-based logic implemented **directly in Spring Boot**. No external dependency.

Example Rules

```
// Sleep insight
if (avgSleep < 6.0) {
    insights.add("⚠️ Your average sleep is below recommended level ("
        + avgSleep + " hrs vs 7 hrs recommended)");
}

// Expense insight
if (weeklySpending > userBudgetThreshold) {
    insights.add("💰 You are overspending this week. Budget: ₹"
        + threshold + ", Spent: ₹" + weeklySpending);
}

// Habit insight
```

```
if (habitCompletionRate < 50) {
    insights.add("📉 Low habit consistency this week ("
        + habitCompletionRate + "%). Try completing at least 3 habits daily.");
}

// Water intake
if (avgWaterIntake < 2.5) {
    insights.add("💧 Water intake is below target ("
        + avgWaterIntake + "L vs 2.5L recommended)");
}

// Positive reinforcement
if (currentStreak > previousStreak) {
    insights.add("🔥 Great momentum! Your activity streak is growing.");
}
```

Rule Categories

Category	Threshold	Insight Type
Sleep	`avgSleep < 6`	Warning
Sleep	`avgSleep >= 8`	Positive
Steps	`avgSteps < 5000`	Warning
Steps	`avgSteps > 10000`	Positive
Spending	`weeklySpending > budget`	Warning
Spending	`weeklySpending < budget * 0.8`	Positive
Habits	`completionRate < 50%`	Warning
Habits	`completionRate > 80%`	Positive
Water	`avgWater < 2.5L`	Warning
Mood	`dominant mood = Tired/Stressed`	Suggestion

Benefits of Rule-Based Phase

- ⚡ Fast — no API call latency
- 🔓 Always available — no external dependency
- 🎯 Deterministic — same input = same output
- 💰 Free — no LLM API costs

---

6. Phase 2 — AI-Based Insight Engine (LLM)

Description

Uses an LLM to generate **personalized, context-aware insights** that go beyond simple threshold checks.

Capabilities

Rule-Based (Phase 1)	LLM-Based (Phase 2)
"Sleep is below 6 hrs"	"Your sleep has been declining since Tuesday. Consider reducing screen time before bed — your journal mentioned late-night coding sessions."
"You are overspending"	"Your food spending spiked 40% this week. Most of it was on dining out. Cooking 2 meals at home could save ~₹800."
Deterministic, template	Context-aware, natural language

LLM Prompt Structure

```
System: You are a lifestyle coach AI. You analyze user data and provide
actionable, encouraging insights. Be specific and reference actual numbers.

User Context:
- Average sleep this week: 5.5 hours (down from 7.2 last week)
- Average steps: 3,000 (target: 8,000)
- Weekly spending: ₹3,200 (budget: ₹2,500)
- Top expense: Food (₹1,800)
- Habit completion: 60%
- Dominant mood: Tired (3/7 days)
- Sleep trend: declining

Generate 3-4 specific, actionable insights based on this data.
```

Example Queries and Responses

User Question	AI Response
"Why am I feeling tired?"	"Based on your data, your sleep averaged 5.5 hrs this week — down from 7.2 hrs last week. Combined with only 3,000 daily steps, your energy levels are likely affected. Try setting a 10 PM bedtime alarm."
"Where am I spending too much?"	"Food is your top category at ₹1,800 (56% of weekly spend). You spent ₹900 on dining out alone. Cooking 3 meals at home this week could save ₹600."
"How can I improve my routine?"	"Start with your biggest gap: sleep. You're averaging 1.5 hrs below target. Your habit completion drops to 40% on days you sleep under 6 hrs — fixing sleep will likely improve everything else."

7. Phase 3 — RAG Chatbot (Retrieval-Augmented Generation)

Description

Implements **Retrieval-Augmented Generation** for conversational AI that can answer questions using the user's historical data.

RAG Flow

```
1. User sends query: "How was my week?"
        ↓
2. Backend prepares context:
   • User's weekly data summary
   • Recent journal entries (summarized)
   • Current goals/challenges
        ↓
3. AI Service receives:
   {
     "question": "How was my week?",
     "context": {
       "weekSummary": { ... },
       "recentMoods": ["Happy", "Tired", ...],
       "activeGoals": [...]
     }
   }
        ↓
4. (Optional) Vector DB retrieval:
   • Embed the question
   • Find semantically similar past data points
   • Add to context
        ↓
5. LLM generates response using retrieved context
        ↓
6. Response returned to user
```

Vector Database (Optional Advanced Feature)

```
┌──────────────────────────┬──────────────────────────┐
│  MySQL + MongoDB         │  Vector DB               │
│  (existing)              │  (new — Phase 3)         │
│                          │                          │
│  Structured data:        │  Embeddings:             │
│  • users, expenses       │  • journal embeddings    │
│  • daily logs, journals  │  • daily log summaries   │
│                          │  • insight history       │
│  Used for: aggregation   │  Used for: semantic      │
│  and direct queries      │  search and retrieval    │
└──────────────────────────┴──────────────────────────┘
```

---

8. AI Service API Contracts

8.1 Generate Insights

`POST /ai/insights`

Request (from Spring Boot → AI Service):

```
{
  "summary": {
    "avgSleep": 5.5,
    "avgSteps": 3000,
    "weeklySpending": 3200,
    "habitCompletionRate": 60,
    "moodDistribution": {"Happy": 3, "Tired": 2, "Neutral": 2},
    "sleepTrend": "declining"
  }
}
```

Response:

```
{
  "insights": [
    {
      "type": "warning",
      "category": "sleep",
      "message": "Your sleep has been declining all week..."
    },
    {
      "type": "suggestion",
      "category": "expense",
      "message": "Try meal prepping on Sunday to reduce food expenses..."
    }
  ]
}
```

8.2 Chat API

`POST /ai/chat`

Request:

```
{
  "question": "How can I improve my sleep?",
  "context": {
    "avgSleep": 5.5,
```

```
    "sleepTrend": "declining",
    "recentMoods": ["Tired", "Tired", "Neutral"],
    "recentJournalSummary": "User mentioned late-night work sessions"
  }
}
```

Response:

```
{
  "answer": "Based on your data, here are 3 specific steps...",
  "confidence": 0.87,
  "sources": ["sleep_data", "mood_data", "journal_data"]
}
```

---

9. Failure Handling

Fallback Strategy

```
AI Service Available?
    ├── YES → Use LLM-generated insights (Phase 2/3)
    └── NO  → Fallback to rule-based insights (Phase 1)
              Return default message: "AI insights are temporarily unavailable.
              Here are your automated insights based on your data patterns."
```

Implementation:

```java
public List<Insight> getInsights(String userId) {
    UserSummary summary = aggregateUserData(userId);

    // Always generate rule-based insights
    List<Insight> ruleInsights = ruleEngine.generate(summary);

    try {
        // Try AI-powered insights
        List<Insight> aiInsights = aiServiceClient.getInsights(summary);
        return mergeInsights(ruleInsights, aiInsights);
    } catch (Exception e) {
        log.warn("AI service unavailable, using rule-based fallback");
        return ruleInsights;
    }
}
```

Key Rules:

- AI calls should be **asynchronous** where possible
- Cache repeated insights (same data = same insight for short period)
- Rate limit AI calls per user (avoid abuse)
- Never expose raw user data to AI unnecessarily

---

10. Security Considerations

Concern	Mitigation
Raw journal content sent to AI	Send summarized/anonymized content only
AI service exposed publicly	Internal-only endpoint, not accessible from frontend
Prompt injection	Sanitize user questions before sending to LLM
API key exposure	Store AI API keys in environment variables, never in code
Data retention	AI service should not persist user data

---

11. Performance Considerations

Strategy	Details
Caching	Cache insights for 1 hour (same data = same insights)
Async calls	AI insight generation doesn't block dashboard loading
Rate limiting	Max 10 AI chat requests per user per hour
Timeout	10-second timeout on AI service calls, fallback to rules
Batch processing	Pre-generate insights during off-peak hours (optional)

---

12. Future Enhancements

- 🧠 Personalized recommendations using user history patterns
- 📈 Predictive analytics (habit trends, expense forecasting)
- 🔄 Continuous learning from user feedback on insights
- 🗣️ Voice input for daily logging
- 📊 Comparative analytics ("You're in the top 20% for steps this week")

Summary: What AI Does and Does NOT Do

AI Handles:

```
☑ Generating natural language insights from aggregated data
☑ Answering user questions about their lifestyle
```

```
☑ Finding patterns humans might miss
☑ Making recommendations feel personal and actionable
```

AI Does NOT Handle:

```
❌ Storing or accessing data directly
❌ Authentication or authorization
❌ Data aggregation or computation (backend does this)
❌ Being the only source of insights (rule-based always available)
```

> **The platform is the product. AI makes it intelligent.**